



—

# School of DevOps

## Git Fundamentals

BOGDAN COSTEA – DEVOPS ENGINEER – CLD



# Agenda

1. WHAT ARE VERSION CONTROL SYSTEMS AND WHY SHOULD I CARE
2. GIT INTRODUCTION
3. BASIC COMMANDS
4. ADVANCED COMMANDS
5. HOW TO INSTALL GIT
6. DEMO

1

# What are version control systems and why should I care





# Version Control Systems

Systems that help with tracking changes done to code, documents or other collections of information.

- ▶ Allows for tracking change history
- ▶ Enables team collaboration

# Without VCS

We would need to manually keep track of different files that represent the same content



**FILE.TXT**

First version of our file



**FILE.TXT**  
**FILE.TXT2**

Second version of our  
file



**FILE.TXT**  
**FILE.TXT2**  
**FINALFILE.TXT**

Final version of our file

# With VCS

The system keeps track of the changes done to the file, so there's no need to keep different folders/files representing the same thing anymore



FILE.TXT

First version of our file



FILE.TXT

Second version of our  
file



FILE.TXT

Final version of our file





---

## TEAM COLLABORATION

Enables concurrent work.  
Multiple contributors can  
work on the same project  
simultaneously.



---

## TRACEABILITY

Every change can be traced  
and associated with a work  
item (bug, user story, etc).  
Changes are also  
associated with a message  
describing what has been  
modified.



---

## TRACK HISTORY

If mistakes are made,  
developers can easily go  
back to a previous version  
of the files.

# VCS Types

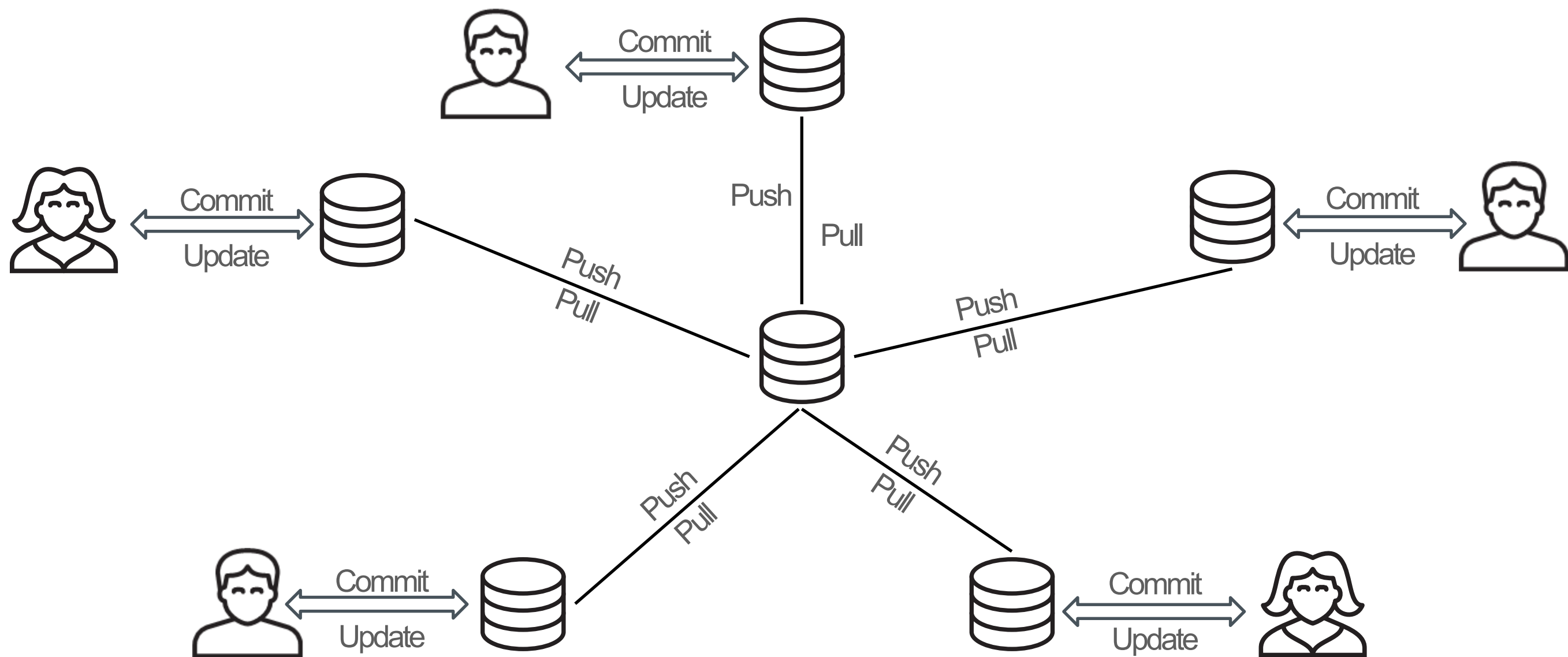
Version Control Systems generally belong to one of the following categories:

- ▶ Distributed: Git, Mercurial
- ▶ Centralized: Team Foundation Server, Subversion



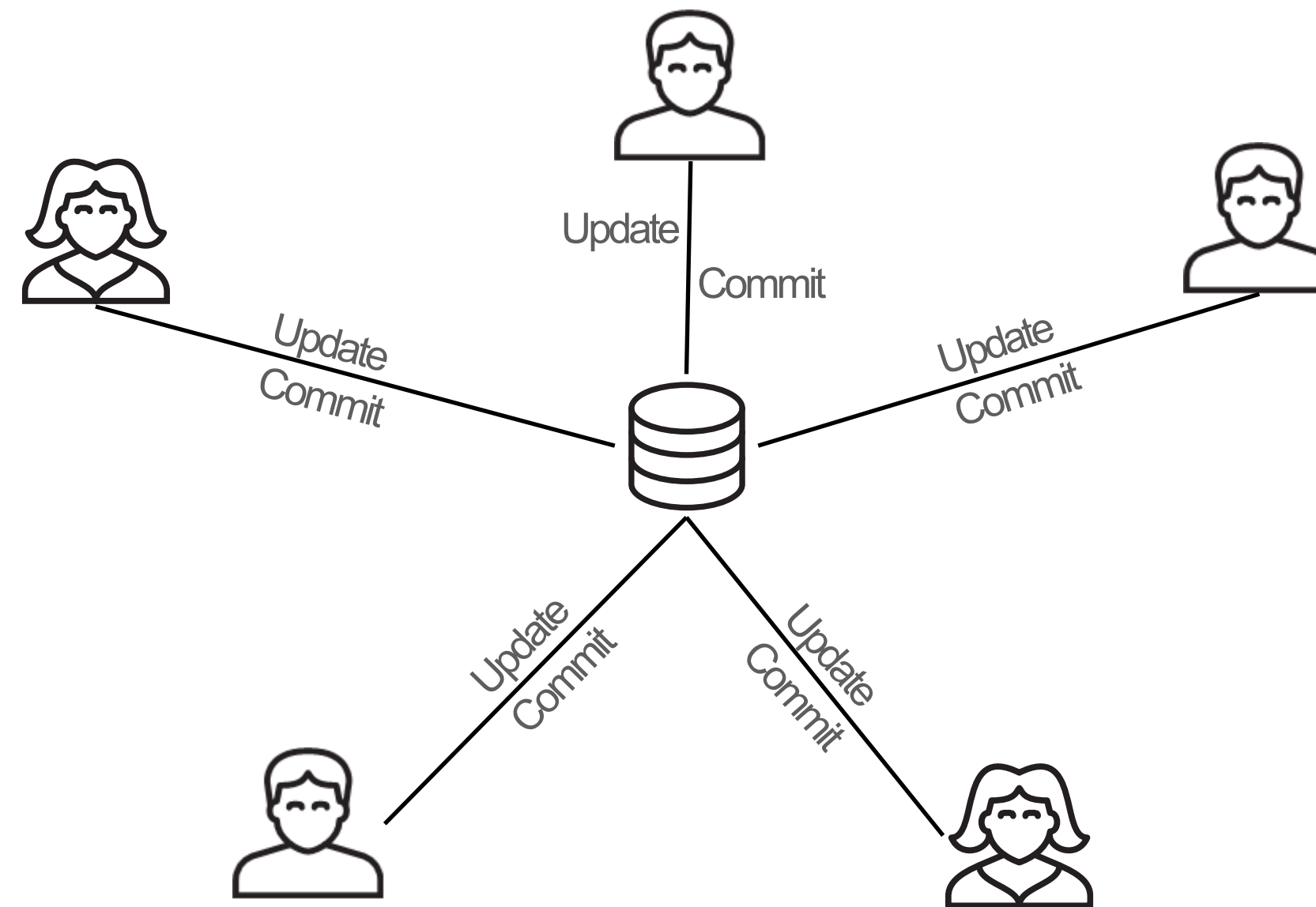
# Distributed VCS

Each contributor has a copy(clone) of the repository



# Centralized VCS

Each contributor synchronizes their changes with the central repository





2

# Git Introduction





# Key Terminology

To understand what Git is and how we can integrate it in our daily work, we need to define a few terms.

- **REPOSITORY**

Frequently called just “repo”, it is the database that contains the code along with all its changes.

- **CLONE**

Generally refers to the process of cloning a repository which will create a local copy of it.

- **ORIGIN**

Represents the remote, primary repository. Generally the remote server where we push/pull our changes to/from.

- **STAGING**

Refers to adding our changes to the staging area where we can do a final check before committing them to the local repo.

- **COMMIT**

When we consider that our changes are done and sharing with the rest of the team we can decide to commit them to the local repository. A commit represents a snapshot(version) of the code.

- **PUSH**

Pushing is the process through which we can send our changes from the local repository to origin.

- **PULL**

Pulling is the process through which we update our local repository with new changes from remote(origin).

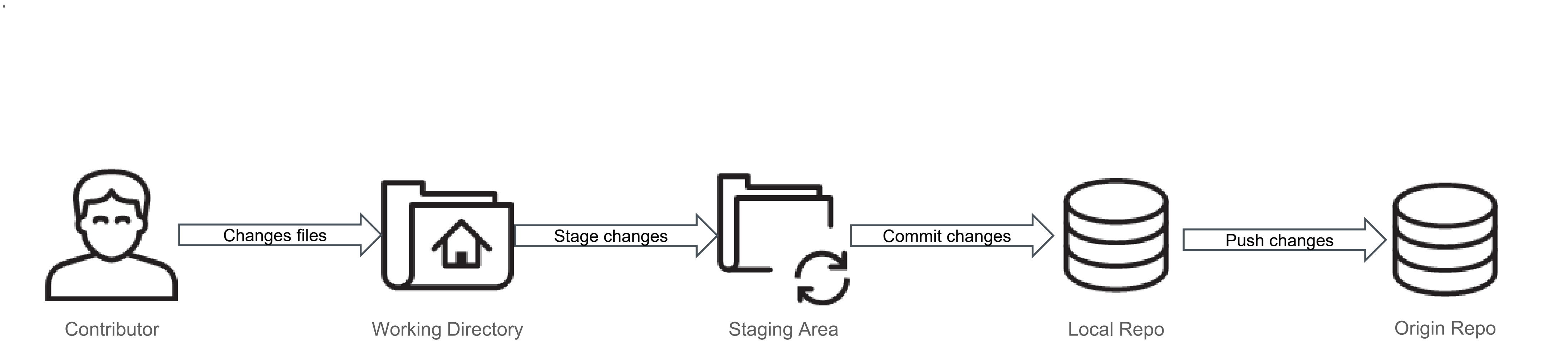
- **BRANCH**

Branches allow us to keep track of multiple different timelines of the code. This is what enables good workflows in a team.

- **MERGE**

Merging is the process through which Git allows for one branch to combine their its history into another.

# Basic Git Workflow



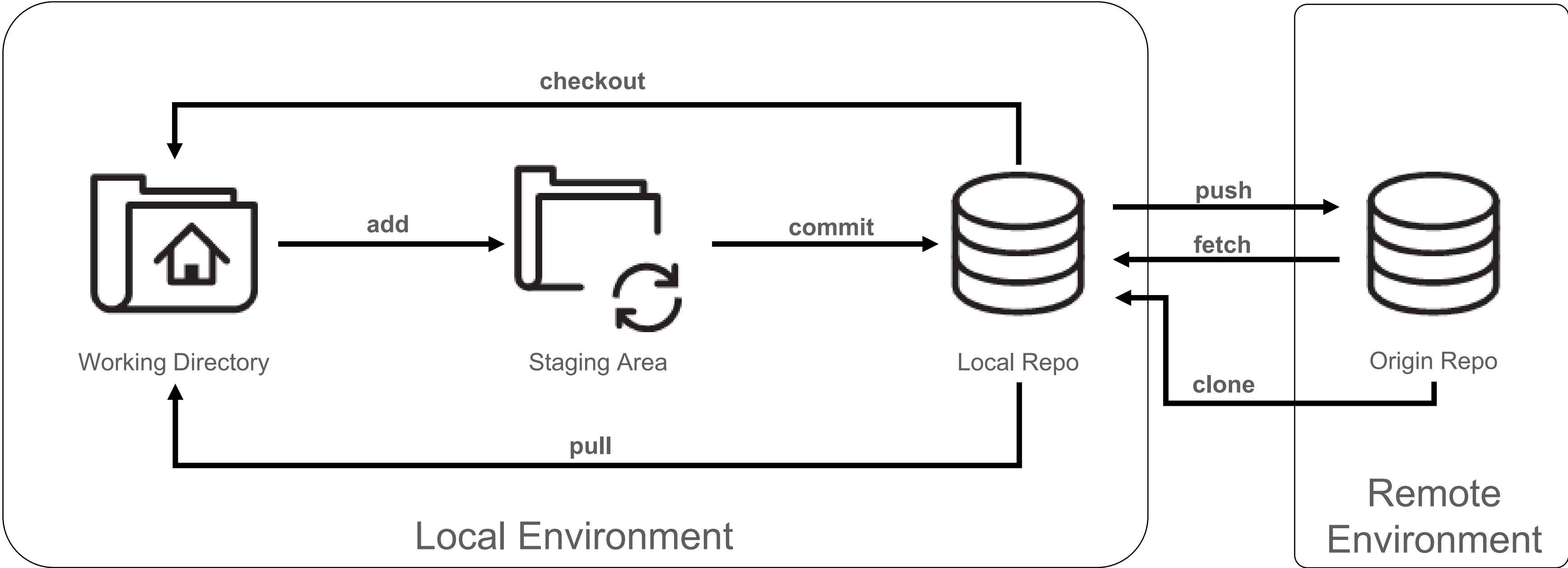
3

# Basic Commands





# Basic Git Commands



# Basic Git Commands

- **GIT INIT**

Initializes a local Git repository.

- **GIT ADD**

Stages local files, adding them from the working directory to the staging area.

- **GIT COMMIT**

Creates a snapshot of the code containing the staged changes.

- **GIT STATUS**

Displays the state of the working directory as well as the staging area.

- **GIT PUSH**

Attempts to send the changes stored in the local repository to the origin.

- **GIT CLONE**

Clones a remote repository locally, creating a copy of it.

- **GIT PULL**

Fetches origin updates and merges them in the working directory.

- **GIT FETCH**

Updates the local repository with the latest changes from origin.

- **GIT CHECKOUT**

Changes the current working branch. Also used to create new branches.

- **GIT MERGE**

Attempts to combine two branches (different histories) into one.

- **GIT STASH**

Can be used to save the current changes locally, without creating a commit. Useful when something more important needs to be taken care of (Production Issue for example) and we are not ready to commit our changes.

- **GIT RESET**

Used to discard previous commits. Caution: Can also be used to rewrite the history.

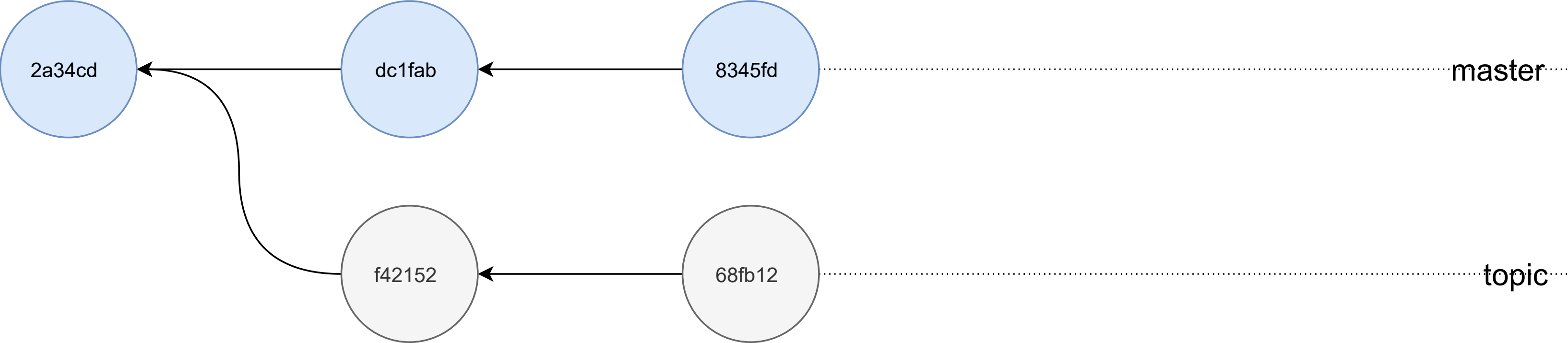


4

# Advanced Commands

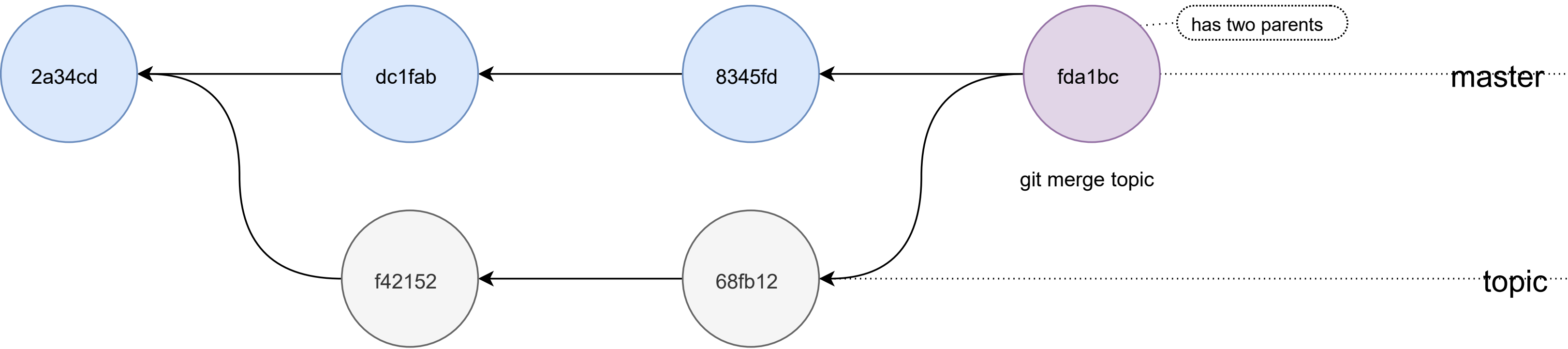


# Initial branches



# Git Merge

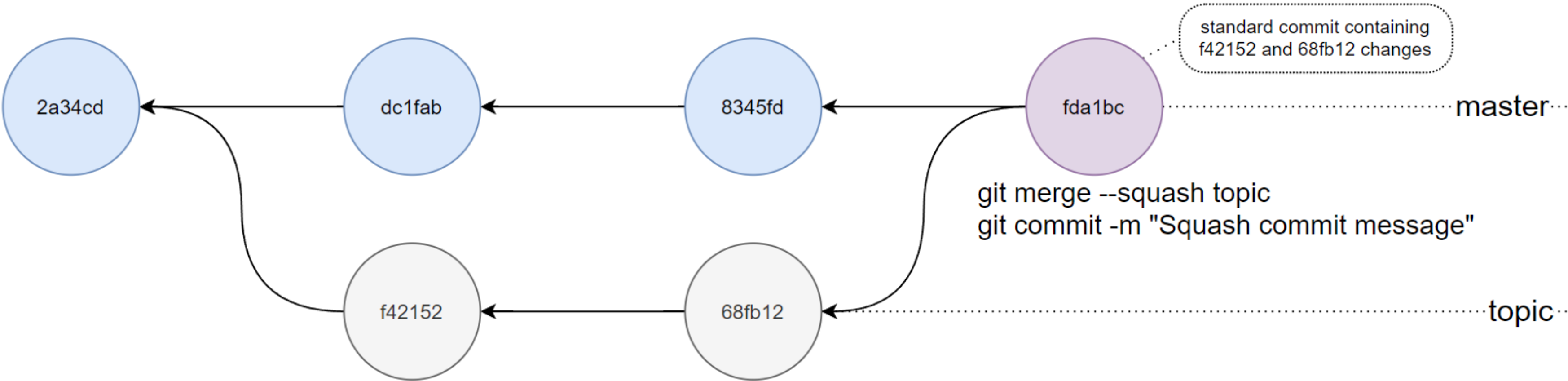
Git merge is a command designed to help with integrating changes from one branch to another in a non-destructive way





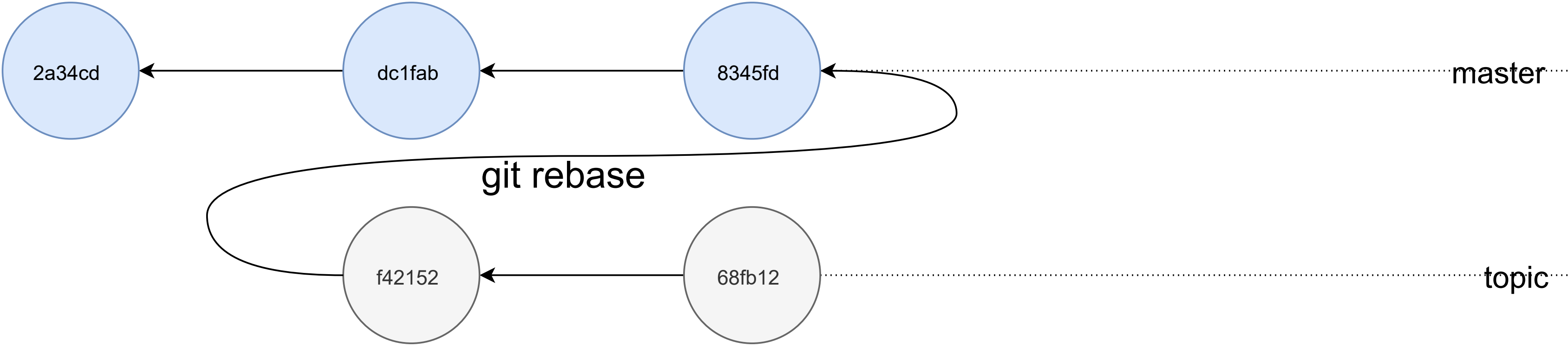
# Git Merge Squash

Git merging with the `--squash` option



# Git Rebase

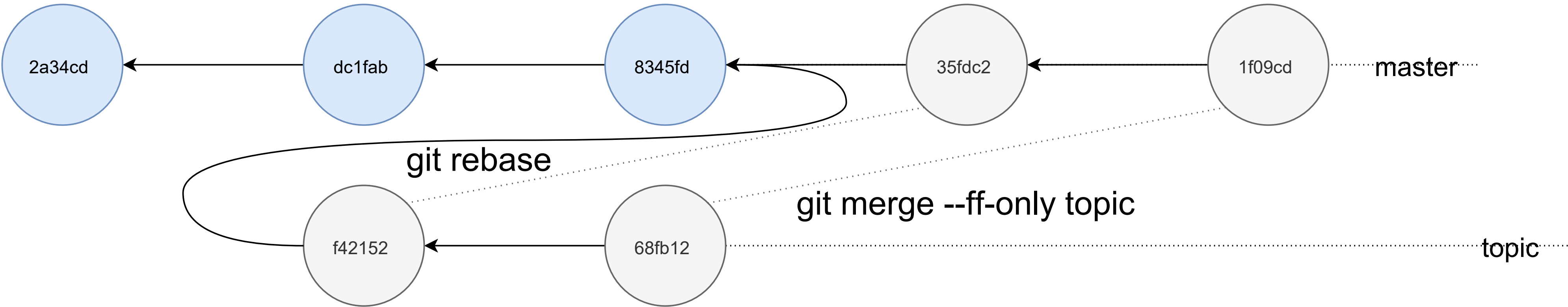
Git rebase is a command that helps moving the base commit of a branch





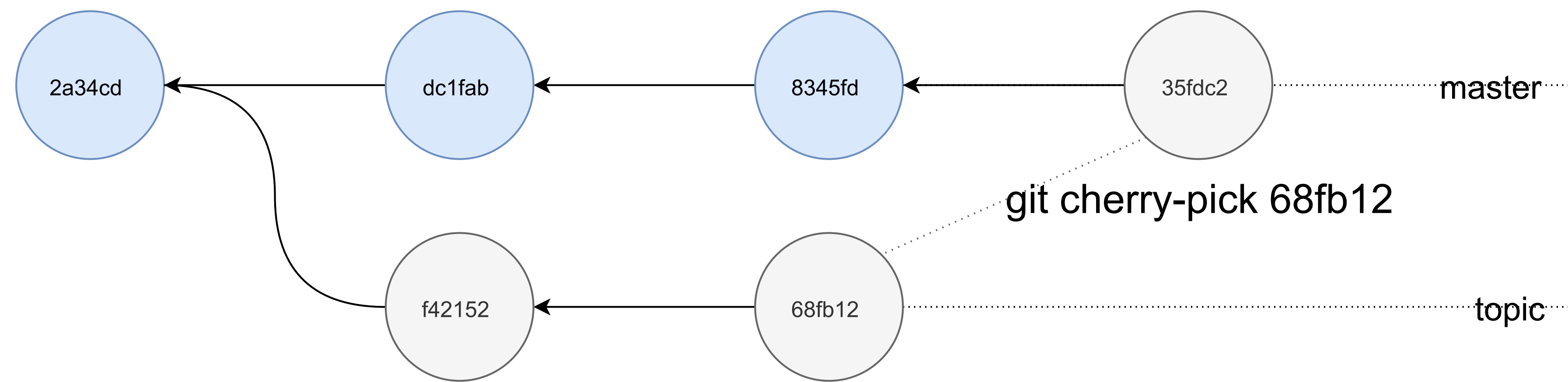
# Git Rebase and Fast Forward

Rebasing then fast-forward merging can help keep a clean history

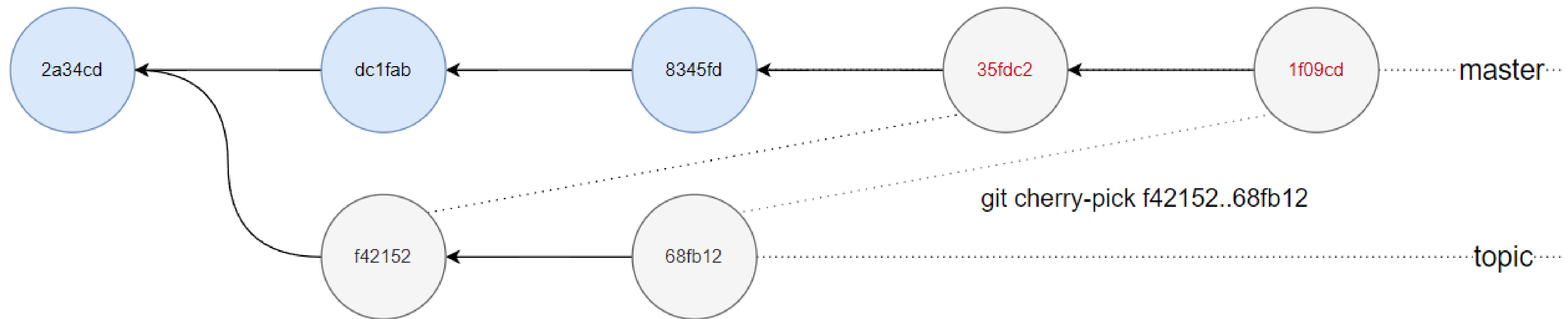


# Git Cherry-Pick

Cherry picking allows for one or multiple commits to be replayed on the target branch



# Git Cherry-Pick multiple commits





5

# How to install git



# Git Installation

There are multiple way you can get Git on your machine.  
Assuming you work on a Windows machine:

- ▶ Download and install from official website: [Download Windows Git \(git-scm.com\)](https://git-scm.com/download/windows)
- ▶ Using Chocolatey Package Manage: `choco install git`
- ▶ For other operating systems, please refer to: [Git - Installing Git \(git-scm.com\)](https://git-scm.com/docs)



# Git Initial Configuration

After installing git, there are some additional mandatory configurations that need to be done. We assume you're using Visual Studio Code as an IDE:

- ▶ `git config --global user.name "FirstName LastName"`
- ▶ `git config --global user.email first.last@endava.com`
- ▶ `git config --global core.editor "code --wait"`
- ▶ `git config --global core.autocrlf true`
- ▶ `git config --global diff.tool vscode`
- ▶ `git config --global difftool.vscode.cmd "code --wait --diff '$LOCAL' '$REMOTE'"`

6

Demo

[Feedback form](#)



